Pontifícia Universidade Católica de Minas Gerais
Instituto de Ciências Exatas e Informática
Algoritmos e Estruturas de Dados II

Trabalho Prático II

Regras Básicas

1. extends Trabalho Prático 01
2. Neste trabalho, nas questões em Java, adicionalmente podem ser utilizados os métodos `int compareTo(String str)`, `boolean equals(String str)`, `String[] split(String str)` e `static String format(String str, Object... args)` da classe `String`, além dos métodos que convertem String para tipos primitivos. Nas questões em C, adicionalmente, podem ser utilizadas as funções `int strcmp(char* str1, char* str2)` e `char* strtok(char *str, const char *delimiters)` de `string.h`, `int sscanf(char* str, char* format, ...)` e `int sprintf(char* buffer, char* format, ...)` de `stdio.h`, além das funções que convertem string para tipos primitivos.

Dataset - Explorando Veículos

Este dataset apresenta uma coleção diversificada de veículos de diferentes fabricantes, categorias e anos, reunindo modelos com variadas características mecânicas e tecnologias de propulsão. A base contempla veículos com diferentes tipos de combustível, transmissões, sistemas de tração, cilindradas e níveis de consumo, permitindo uma visão ampla da evolução e da diversidade do setor automotivo.



Através desses dados, é possível explorar padrões interessantes, como a relação entre características do motor e consumo de combustível, diferenças de eficiência entre categorias de veículos, variações nas

emissões de CO2 e a presença de diferentes tecnologias de transmissão e propulsão ao longo dos anos. Cada registro representa uma configuração única de veículo, contribuindo para um conjunto de dados rico e multifacetado.

Este dataset convida à investigação e análise de dados, estimulando o desenvolvimento de soluções que simulam aplicações do mundo real, como sistemas de busca, recomendação e organização de informações. O dataset permite explorar conceitos importantes como organização e tipagem de dados, modelagem de entidades do mundo real, manipulação de estruturas (listas, registros, etc.), bem como consultas, filtragens e ordenações.

A tabela a seguir apresenta o dicionário de dados referente ao dataset. O dataset está disponível no arquivo VEICULOS.CSV.

Classe + Registro

O Verde pressupõe a presença do arquivo do dataset (VEICULOS.CSV na pasta /tmp/, de onde seu programa fará a leitura. Portanto, o arquivo do dataset deve ser copiado para a pasta /tmp/ em seu ambiente Linux. Quando reiniciamos o Linux, ele normalmente apaga os arquivos existentes na pasta /tmp/.

Neste trabalho, voce deverá realizar a modelagem de dados de um sistema que representa uma coleção de veículos. O objetivo e definir estruturas de dados que permitam representar adequadamente as informações de cada veículo, servindo como base para etapas futuras do trabalho, nas quais serao aplicados algoritmos e estruturas de dados como listas, filas, árvores, além de operacões de pesquisa e ordenação.

Observações

- A modelagem deve ser compatível tanto com linguagem C quanto Java
- É vedada a dependência de bibliotecas externas
- Priorize clareza, organização e boa definição dos tipos
- Pense na reutilização das estruturas em etapas futuras

Deve ser observado que a modelagem será utilizada para:

- leitura completa do dataset
- ordenação por diferentes critérios (campos simples e compostos)
- busca (sequencial e binária)
- uso de estruturas como listas, pilhas, filas e árvores

Tabela 1: Dicionário de dados do dataset de veículos

Campo	Descrição	Tipo/Formato
`id`	Identificador único do veículo. Os valores são inteiros e não seguem ordem sequencial no dataset.	Inteiro
`marca`	Fabricante ou marca comercial do veículo.	String
`modelo`	Nome ou identificação comercial do modelo/configuração do veículo.	String
`ano`	Ano-modelo do veículo.	Inteiro – `AAAA`
`categoria`	Categoria à qual o veículo pertence, como carros compactos, utilitários esportivos, picapes ou veículos de dois lugares.	Categórico / String
`combustivel`	Tipo ou tipos de combustível/energia utilizados pelo veículo. Pode possuir mais de um valor, separados por `;`, como `Gasoline;Electricity` ou `Gasoline;E85`.	Categórico multivalorado / String
`cilindros`	Número de cilindros do motor a combustão. Para veículos puramente elétricos, utiliza-se o valor `0`.	Inteiro
`cilindrada`	Volume total dos cilindros do motor, expresso em litros. Para veículos puramente elétricos, utiliza-se `0.0`.	Real, em litros
`transmissao`	Tipo de transmissão do veículo. Os valores são normalizados em `Automatic`, `Manual`, `CVT` e `Automated Manual`.	Categórico / String
`tracao`	Sistema de tração do veículo, indicando quais rodas recebem força motriz, como dianteira, traseira, integral ou 4x4.	Categórico / String
`consumo_cidade`	Eficiência energética do veículo em condições urbanas. Para veículos elétricos, representa o valor equivalente obtido a partir do MPGe.	Real, em km/L ou km/L equivalente
`consumo_estrada`	Eficiência energética do veículo em condições rodoviárias. Para veículos elétricos, representa o valor equivalente obtido a partir do MPGe.	Real, em km/L ou km/L equivalente
`co2`	Emissão direta de dióxido de carbono do veículo. Para veículos puramente elétricos, o valor é `0`.	Real, em g/km
`turbo`	Indica se o veículo utiliza sistema de sobrealimentação por turbocompressor.	Booleano – `true` ou `false`
`data_registro`	Data associada à criação ou liberação do registro do veículo na fonte de dados. Não corresponde necessariamente à data de fabricação do veículo.	Data — AAAA-MM-DD

Exercícios

1.  **Modelagem em Java**: Crie os tipos solicitados seguindo todas as regras apresentadas no slide unidade00l_conceitosBasicos_introducaoOO.pdf. Lembre-se:

   - os tipos são criados usando classes
   - atributos devem ser privados (encapsulamento!!!)
   - implemente os métodos *get* para os atributos
   - avalie se deve ou não implementar os métodos *set* para os atributos

.

A figura 1 apresenta o diagrama de classes para a modelagem em Java. Você deve seguir este diagrama como referência para a implementação do seu código.

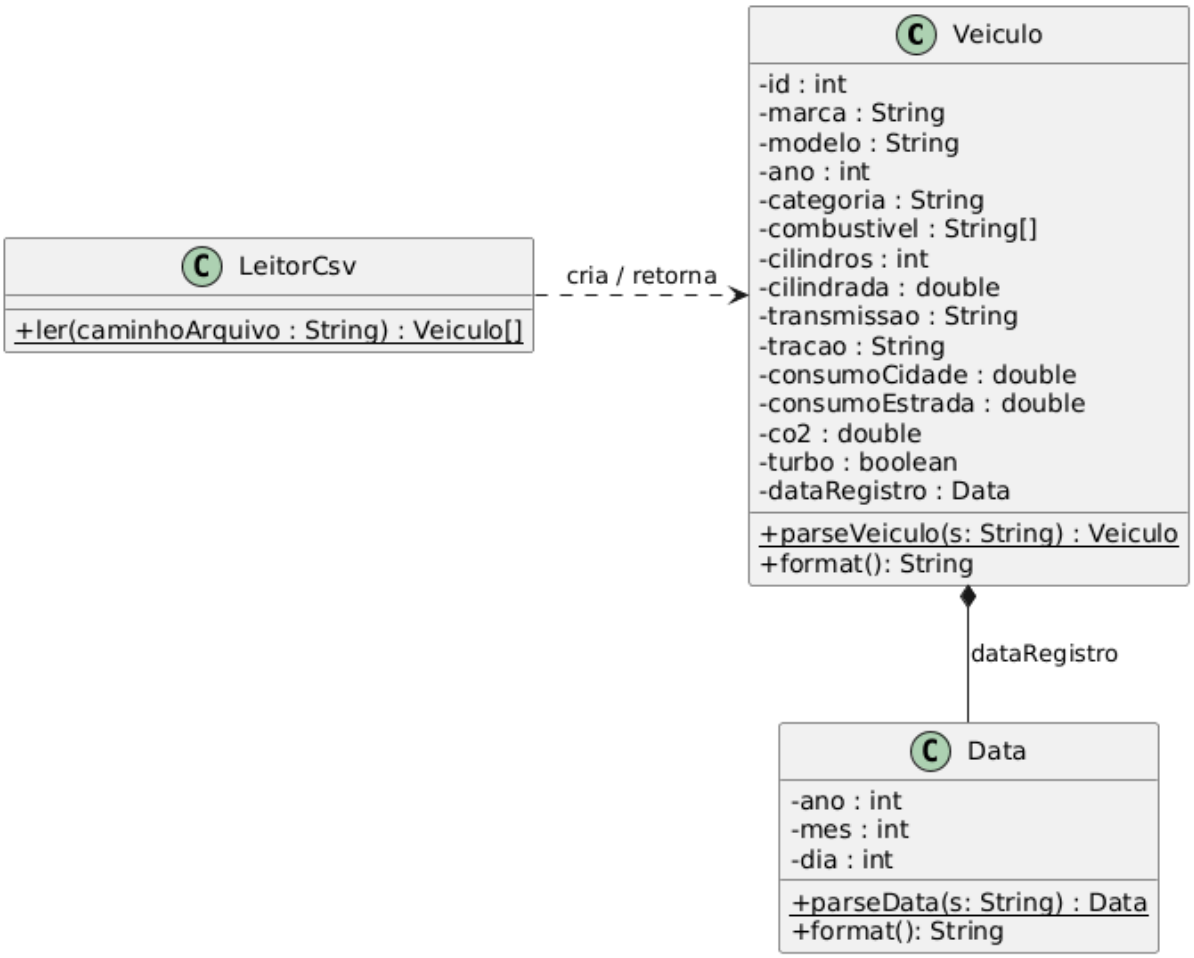


Figura 1: Diagrama de classes da modelagem em Java

O método `format()` da classe `Data` deve retornar uma String que representa a data no formato DD/MM/YYYY. Já o método `format()` da classe `Veiculo` deve retornar uma String que representa o veículo no formato:

```
[id ## marca ## modelo ## ano ## categoria ## [combustivel] ## cilindros ##
cilindrada ## transmissao ## tracao ## consumoCidade ## consumoEstrada ## co2
## turbo ## dataRegistro]
```

Nessa questão, a entrada de dados (entrada padrão) contém um conjunto de números inteiros organizados um por linha, sendo que cada número representa o `id` de um veículo. A última linha da entrada de dados contém -1, indicando o fim.

Seu programa devrá imprimir na saída padrão, em cada linha, o veículo correspondente ao `id` lido na entrada. Portanto, para cada `id` lido na entrada, deverá ser feira uma pesquisa sequencial para encontrar o veículo correspondente.

2. **Modelagem em C**: Repita o exercício Modelagem em Java, agora na linguagem C. Lembre-se que os tipos são criados usando `struct`. A figura 2 apresenta o diagrama de classes para a modelagem em C realizando as devidas adaptações.

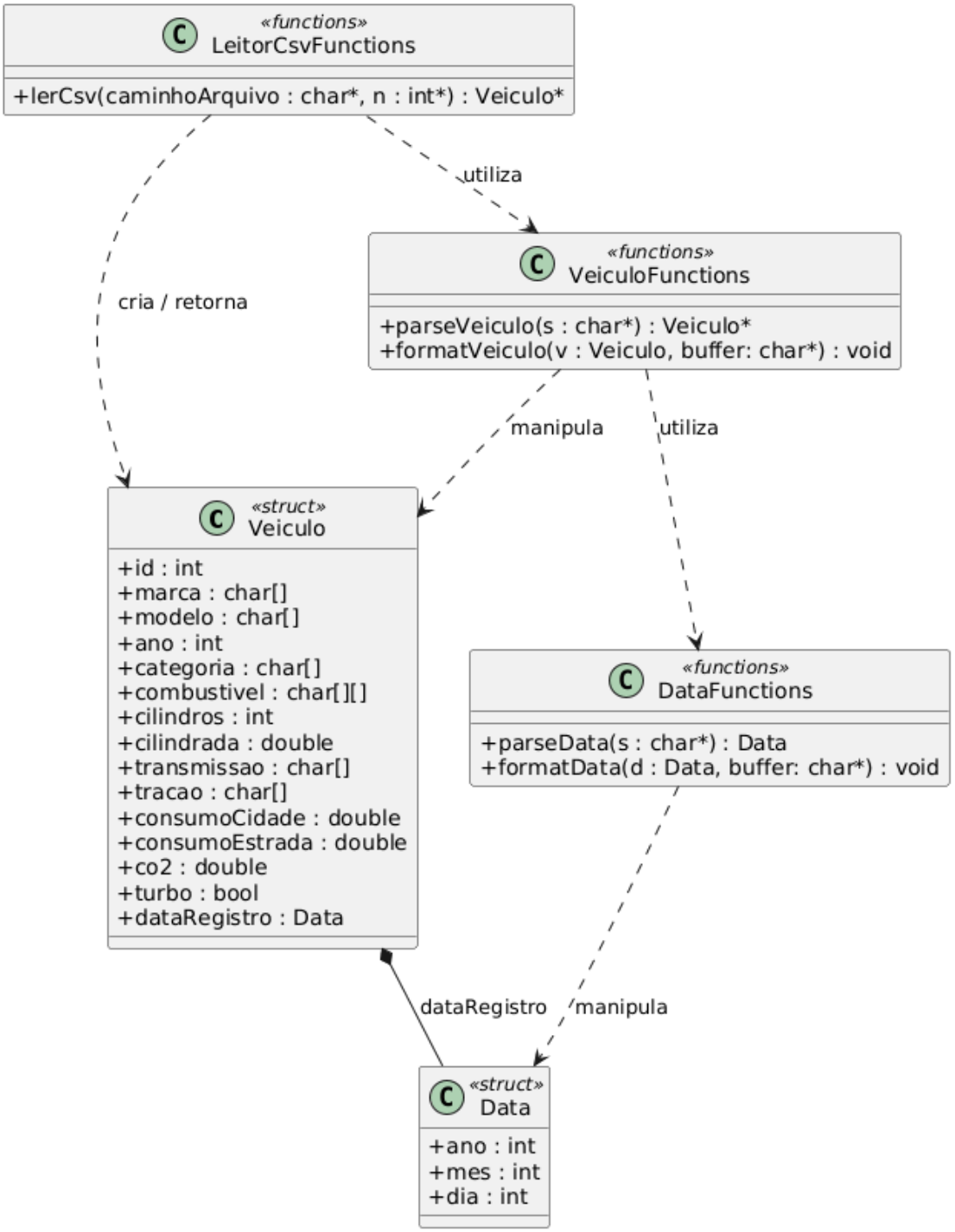


Figura 2: Diagrama de classes da modelagem em C

Pesquisa e Ordenação

3.  **Ordenação por Seleção em C**: Usando arranjos, implemente o algoritmo de ordenação por seleção considerando que a chave de pesquisa é o atributo **modelo**. A entrada e a saída padrão são iguais as da primeira questão, contudo, a saída corresponde aos registros ordenados.

4.  **Ordenação por Inserção em Java**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo de Inserção. A chave de pesquisa é o atributo **marca**.

5.  **Ordenação por Counting Sort em C**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo Counting Sort, fazendo com que a chave de pesquisa seja o atributo **cilindros**.

6.  **Ordenação por Radixsort em C**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo Radixsort, fazendo com que a chave de pesquisa seja o atributo **ano**.

7.  **Ordenação por Bucketsort em Java**: Repita a questão de Ordenação por Seleção, contudo, usando o algoritmo Bucketsort, fazendo com que a chave de pesquisa seja o atributo **cilindrada** normalizada pelo valor 8.1, usando 10 baldes e ordenação por inserção para cada balde.

8.  **Pesquisa Binária em C**: Implemente o algoritmo de pesquisa binária considerando a chave primária de pesquisa o atributo **modelo**. Para a ordenação, use o algoritmo de seleção implementado anteriormente. A entrada padrão é composta por duas partes onde a primeira é igual a entrada da primeira questão. O conjunto de veículos da primeira parte deve ser armazenado em um arranjo separado, que servirá de base para as pesquisas a serem realizadas. A segunda parte da entrada de dados é composta por várias linhas. Cada uma possui um elemento que deve ser pesquisado no arranjo. A última linha terá a palavra FIM. A saída padrão será composta por várias linhas contendo as palavras SIM/NAO para indicar se existe cada um dos elementos pesquisados.

Estruturas Lineares

9.  **Lista com Alocação Sequencial em Java**: Crie uma Lista de registros baseada na de inteiros vista na sala de aula. Sua lista deve conter todos os atributos e métodos existentes na lista de inteiros, contudo, adaptados para a classe `Veiculos`. Lembre-se que, na verdade, temos uma lista de ponteiros (ou referências) e cada um deles aponta para um registo. Neste exercício, faremos inserções, remoções e mostraremos os elementos de nossa lista.

Os métodos de inserir e remover devem operar conforme descrito a seguir, respeitando parâmetros e retornos. Primeiro, o `void inserirInicio(Veiculo veiculo)` insere um registro na primeira posição da Lista e remaneja os demais. Segundo, o `void inserir(Veiculo veiculo, int posicao)` insere um registro na posição *posicao* da Lista, onde *posicao* $< n$ e n é o número de registros cadastrados. Em seguida, esse método remaneja os demais registros. O `void inserirFim(Veiculo veiculo)` insere um registro na última posição da Lista. O `Veiculo removerInicio()` remove e retorna o primeiro registro cadastrado na Lista e remaneja os demais. O `Veiculo remover(int posicao)` remove e retorna o registro cadastrado na posição informada da Lista e remaneja os demais. O `Veiculo removerFim()` remove e retorna o último registro cadastrado na lista.

A entrada padrão é composta por duas partes. A primeira parte é igual a entrada da primeira questão e os registros presentes devem ser inseridos ao final da Lista. As demais linhas correspondem a segunda parte. A primeira linha da segunda parte é composta de um número inteiro n indicando a quantidade de registros a serem inseridos/removidos. Nas próximas n linhas, tem-se n comandos de inserção/remoção a serem processados neste exercício. Cada uma dessas linhas tem uma palavra de comando: `II` inserir no início, `I*` inserir na posição informada, `IF` inserir no fim, `RI` remover no início, `R*` remover na posição informada e `RF` remover no fim. No caso dos comandos de inserção, temos também o identificador do registro a ser inserido. No caso dos comandos de inserção na posição informada, a posição fica imediatamente após a palavra de comando. A saída padrão tem uma linha para cada registro removido, sendo que essa informação será constituída pela palavra "(R)" e os atributos **marca modelo** do registro removido. No final, a saída mostra os registros presentes na Lista do primeitro ao último.

10. **Fila Circular com Alocação Sequencial em C**: Crie uma Fila Circular de registros. A fila deve ter tamanho cinco (capacidade máxima). A entrada padrão será como a da questão Lista com Alocação Sequencial, contudo, teremos apenas os comandos `I` para inserir (enfileirar) e `R` para remover (desenfileirar). Quando o programa tiver que inserir um registro e a fila estiver cheia, antes, ele deve fazer uma remoção.

    Para cada registro removido da fila, a saída padrão apresenta a palavra "(R)" e os atributos **marca modelo** do registro removido. No final, a saída mostra os registros presentes na fila do primeitro ao último.

11. **Lista com Alocação Flexível em C**: Refaça a Questão "Lista com Alocação Sequencial" usando lista simplesmente encadeada.

12. **Pilha com Alocação Flexível em Java**: Crie uma Pilha simplesmente encadeada de registros. Neste exercício, faremos inserções, remoções e mostraremos os elementos de nossa pilha a partir do topo. A entrada e a saída padrão serão como as da questão "Lista com Alocação

Sequencial", contudo, teremos apenas os comandos `I` para inserir (empilhar) e `R` para remover (desempilhar).

13.  **Lista Dupla com Alocação Flexível em Java**: Refaça a Questão "Lista com Alocação Sequencial" usando lista duplamente encadeada.